

Basic TB vs UVM map

You already know how a small block testbench wiggles pins, prints values, and checks results in one module

Six pairs to memorize

- Think in six mappings
- Pin wiggle in an initial block becomes a driver behind a virtual interface
- Hard-coded vectors and nested tasks become sequences feeding a sequencer
- A display peek in the same initial becomes a monitor on an analysis path
- Inline expect becomes a scoreboard comparing predict versus actual on transactions
- One flat testbench module becomes test, environment, and agents around the DUT

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Basic TB vs UVM map

INTERACTIVE TOOL

Basic TB vs UVM map

Map **wiggle-the-pins** testbenches onto UVM roles: sequences, agents, drivers, monitors, and scoreboards. Starter: **UART TX byte** — pin drive already linked to the driver.

Starter example: UART TX byte with pin wiggle ↔ driver selected and marked mapped. Click other pairs, Mark mapped, or Demo all.

Load starter example

Challenges 0 / 22 cleared

Quiz: goal: This lab's main idea is...

☐ map classic pin TB roles onto UVM components

☐ compile Accellera UVM in the browser

☐ synthesize a UART VIP

☐ replace all \$display with SVA

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

Pins → transactions

UVM moves intent from edges to items.

Split roles

Drive, watch, and check are separate components.

Agent

Driver + monitor + sequencer around one interface.

Reuse

Factory and VIP beat copy-paste TBs.

TB vs UVM map lab starter

learn_uvm2017 — tb vs uvm map

Real UVM literacy

```
esl - module02-tb-vs-uvm-map/examples/navigation

# Track A - real Unix
# real shell session (Track A - classic vs UVM map)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module02-tb-vs-uvm-map

$ ls examples/classic-vs-uvm/
map.txt

$ cat examples/classic-vs-uvm/map.txt
Classic testbench • UVM map (literacy - six pairs)

1. Pin wiggle      • Driver + virtual interface
Classic: initial / task drives DUT pins with delays
UVM:      driver pulls seq_items; drives via vif

2. Hard-coded vectors • Sequence + sequencer
Classic: fixed lists / nested tasks per scenario
UVM:      sequences generate items; sequencer → driver

3. $display peak    • Monitor + analysis
Classic: TB reads wires and prints in same block
UVM:      passive monitor → analysis port / TLM

4. Inline expect    • Scoreboard
Classic: if (got == exp) $error in stimulus block
UVM:      predict vs actual compare on transactions

5. Flat module tb    • Env + agent(s)
Classic: one wrapper - clock, stim, check, $finish
UVM:      test → env → agent (sqz/drv/mon) → DUT

6. Copy-paste TB     • Factory / VIP reuse
Classic: fork whole tb.v for next block / protocol
UVM:      type overrides + packaged agents (VIP)

UART vignette (browser lab starter):
send_byte task      • uart sequence item
$display on tx       • uart monitor item
if (rx == byte)     • scoreboard byte compare

$ head -20 ../learn_uvm2017_sv_verilator/docs/MODULE3.md
# Module 3: UVM Basics

**Goal**: Master UVM class hierarchy and phases

## Overview

This module introduces the Universal Verification Methodology (UVM) using SystemVerilog. You'll learn the fundamental
UVM concepts, class hierarchy, phases, and how to structure UVM testbenches. This module establishes the foundation for
all advanced UVM concepts.

### Examples and Code Structure

This module includes comprehensive examples and testbenches located in the 'module3/' directory:

...
modules/
├── examples/          # SystemVerilog UVM examples
│   ├── class_hierarchy/ # UVM class hierarchy examples
│   ├── phases/         # UVM phases examples
│   ├── reporting/      # UVM reporting examples
│   ├── configdb/       # ConfigDB examples
│   └── factory/        # Factory pattern examples
```

Real shell — classic vs UVM map

Pitfalls to watch

- Do not treat UVM as magic that replaces thinking, you still need stimulus, observation
- Do not map only the driver and skip the monitor
- Do not assume every project needs every UVM feature
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser
- On real UVM, restate the six mappings without looking at the cheat sheet
- When you are ready